Fullstack Development

Preflight project - deployment

Github Repo

Local machine

Clear your dev environment

- Remove all containers
  - Check with `docker ps -a`
  - `docker rm -f CONTAINER`
- Remove volumes
  - `docker volume prune -a`
- Remove all image cache
  - `docker image prune -a`
- Remove unused networks
  - `docker network prune`

Setup

- `git clone https://github.com/fullstack-69/pf-deploy.git`
  - *Better yet, fork and clone this repo*
- `cd pf-deploy`
- Make `.env` from `.env.example` (Make necessary changes.)
- Take care of `./_entrypoint/init.sh`
  - Windows: Make sure that you save with LF option.
  - Mac/Linux: `chmod +x ./\_entrypoint/init.sh`
- `docker compose up -d --force-recreate`
- Use `docker compose logs` to inspect.

File Permissions Issue

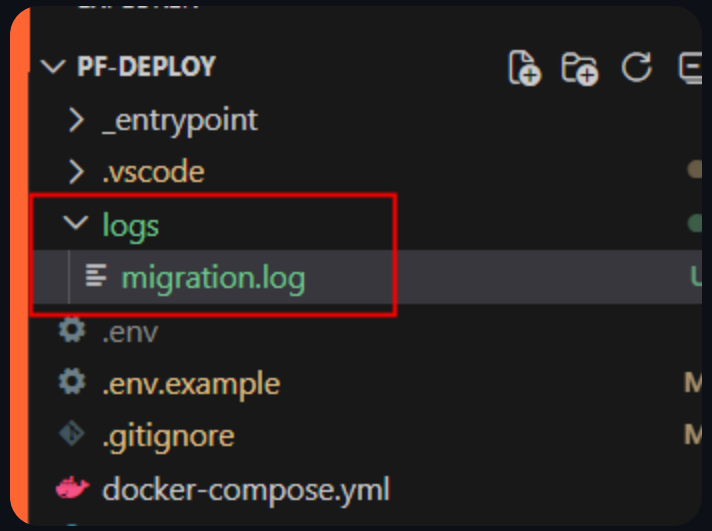
Consider the following volume mapping in your `docker-compose.yml`.

```yaml
name: ${PROJECT_NAME}
services:
  backend:
    volumes:
      - ./logs:/app/logs
```

- On Linux, if you run `docker compose up`, the log files created in the `./logs` folder will have `root` ownership.
  - This is because the container runs as `root` user by default.
  - Your account (non-root) cannot delete these files.

File Permissions Issue

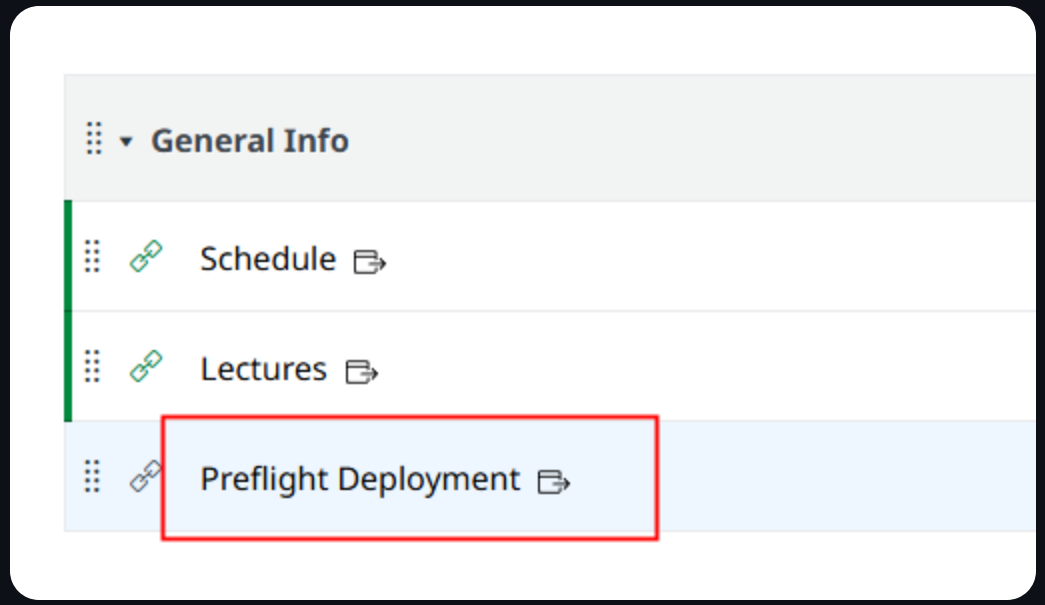
- To fix this, you can create the folder and file beforehand.
- This is why I committed these files in the repo.



> Note: I experimented with `user: ${UID}:${GID}` in the `docker-compose.yml`. It has some issue with corepack.

Remote server

Server info



Server admin

- `ssh USERNAME@10.10.x.x`
- `passwd` to change your password.

Setup

- Clone the `pf-deploy` repo
- `cd pf-deploy`
- `cp .env.example .env`
- `nano .env` and make necessary changes.
  - Use the assigned `FRONTEND_PORT` or else the public url will not work.
  - Save and exit by pressing `Ctrl+S` then `Ctrl+X`.
- Check your container
  - `docker ps | less -S`
- Visit your public `url`.

Redeploy

- Change something in the frontend.
- Rebuild the frontend and redeploy to `dockerhub`.
- `docker compose pull` on the server to update the image.
- `docker compose up -d --force-recreate` to redeploy.

> Docker images will not be updated automatically when you run `docker compose up -d`.

Note on `pull` command

- You might still have container running (from a previous deployment).
- The `:latest` pointer moves from your old local image to the newly downloaded image.
    - The old image is not deleted. It becomes untagged.

> You can use `docker image prune -a` to remove untagged images (after you remove the container that uses it.)

Recap

Topic	Stack
Language	TypeScript
DB	PostgreSQL / Drizzle ORM
Backend	Express
Frontend	React / Vite
Testing	Vitest / Cypress
Deployment	Docker / Nginx

Congratuations!

> Now go and make awesome apps!